

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

程序插桩及 Hook 实验

实验要求：

复现实验一，基于 Windows MyPinTool 或在 Kali 中复现 malloctrace 这个 PinTool，理解 Pin 插桩工具的核心步骤和相关 API，关注 malloc 和 free 函数的输入输出信息。

实验过程：

1. malloctrace 的复现

• 下载与环境准备：

<https://software.intel.com/content/www/us/en/develop/articles/pin-binary-instrumentation-tool-downloads.html> 下载 Linux 版本的 pin 工具，解压后添加到 Linux 虚拟机。

• 编译生成动态链接库：

进入 pin/source/tools/ManualExamples 文件夹，打开终端执行编译命令 make malloctrace.test TARGET=intel64，生成动态链接库 malloctrace.so。

```
(kali@kali)-[~/pin/source/tools/ManualExamples]
$ make malloctrace.test TARGET=intel64
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exce
ptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -D
HOST_IA32E -fPIC -DTARGET_LINUX -fabi-version=2 -faligned-new -I../source/
include/pin -I../source/include/pin/gen -isystem /home/kali/Downloads/pin/
extras/cxx/include -isystem /home/kali/Downloads/pin/extras/crt/include -isystem
/home/kali/Downloads/pin/extras/crt/include/arch-x86_64 -isystem /home/kali/Dow
nloads/pin/extras/crt/include/kernel/uapi -isystem /home/kali/Downloads/pin/extr
as/crt/include/kernel/uapi/asm-x86 -I../extras/components/include -I../
../extras/xed-intel64/include/xed -I../source/tools/Utils -I../source
e/tools/InstLib -O3 -fomit-frame-pointer -fno-strict-aliasing -Wno-dangling-poi
nter -c -o obj-intel64/malloctrace.o malloctrace.cpp
g++ -shared -Wl,--hash-style=sysv ../intel64/runtime/pincrt/crtbeginS.o -Wl,
-Bsymbolic -Wl,--version-script=../source/include/pin/pintool.ver -fabi-
version=2 -o obj-intel64/malloctrace.so obj-intel64/malloctrace.o -L../intel64/runtime/pincrt -L../intel64/lib -L../intel64/lib-ext -L../
../extras/xed-intel64/lib -lpin -lxd ../intel64/runtime/pincrt/crtendS.o
-lpindwarf -ldwarf -ldl-dynamic -nostdlib -lc++ -lc++abi -lm-dynamic -lc-dynamic
-lunwind-dynamic
../pin -t obj-intel64/malloctrace.so -- ../source/tools/Utils/obj-
intel64/cp-pin.exe makefile obj-intel64/malloctrace.makefile.copy \
> obj-intel64/malloctrace.out 2>81
cmp makefile obj-intel64/malloctrace.makefile.copy
rm obj-intel64/malloctrace.makefile.copy
rm obj-intel64/malloctrace.out
```

• 编写并编译测试程序：

编写 C 语言文件，命名为 FirstCpp.c，使用 gcc -o FirstCpp FirstCpp.c 指令将其编译成可执行文件 FirstCpp。

```
#include <stdio.h>

void main()
{
printf("hello world!");
}

First FirstCpp FirstCpp.c
```



FirstCpp.c

```
(kali@kali)-[~/Downloads/testCPP]
$ gcc -o First FirstCpp.c
(kali@kali)-[~/Downloads/testCPP]
```

- 执行并观察结果:

使用 Pin 工具加载 malloctrace.so 对可执行文件插桩并运行, 观察输出结果, 记录 malloc 和 free 函数的调用信息, 包括分配的内存大小和返回的指针地址。

```
(kali@kali)-[~/Downloads/pin-3.18]
$ ./pin -t ./source/tools/ManualExamples/obj-intel64/inscount0.so -- ../testCPP/First
hello world!
```

- 分析输出结果:

输出结果中, malloc(0xXX) 表示分配了 0xXX 字节的内存空间, returns 0xYYYYYYYY 表示返回的内存地址。通过分析这些信息, 可以了解程序运行时的内存分配情况。

malloc(0x5a1)	malloc(0x348)
returns 0x7f719713e180	returns 0x7f7183cb65e0
malloc(0x4a1)	malloc(0x90)
returns 0x7f719713e730	returns 0x7f7183cb6930
malloc(0x10)	malloc(0x410)
returns 0x7f719713ebe0	returns 0x7f7183cb69c0
malloc(0x9d)	malloc(0x1088)
returns 0x7f719713ebf0	returns 0x7f7183cb6dd0
malloc(0x28)	malloc(0x110)
returns 0x7f719713ec90	returns 0x7f7183cb7e60
malloc(0x140)	malloc(0x400)
returns 0x7f719713ecc0	malloc(0x400)
malloc(0x20)	returns 0x55a626b2b2a0
returns 0x7f719713ee00	

2. Inscount0 的复现

- 下载与环境准备:

重复 malloctrace 复现实验中的下载与环境准备步骤, 即从官网 <https://software.intel.com/content/www/us/en/develop/articles/pin-binary-instrumentation-tool-downloads.html> 下载 Linux 版本的 pin 工具, 解压后添加到 Linux 虚拟机。

- 编译生成动态链接库:

在 pin/source/tools/ManualExamples 文件夹下的终端执行 make inscount0.test TARGET=intel64 命令, 生成动态链接库 inscount0.so 。

```

(kali@kali)-[~/pin/source/tools/ManualExamples]
$ make inscount0.test TARGET=intel64
mkdir -p obj-intel64/
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exce
ptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -DHO
ST_IA32E -fPIC -DTARGET_LINUX -fabi-version=2 -faligned-new -I../..../source/inc
lude/pin -I../..../source/include/pin/gen -isystem /home/kali/Downloads/pin/extr
as/cxx/include -isystem /home/kali/Downloads/pin/extras/crt/include -isystem /hom
e/kali/Downloads/pin/extras/crt/include/arch-x86_64 -isystem /home/kali/Downloads
/pin/extras/crt/include/kernel/uapi -isystem /home/kali/Downloads/pin/extras/crt/
include/kernel/uapi/asm-x86 -I../..../extras/components/include -I../..../extra
s/xed-intel64/include/xed -I../..../source/tools/Utils -I../..../source/tools/I
nstLib -O3 -fomit-frame-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o
obj-intel64/inscount0.o inscount0.cpp
g++ -shared -Wl,--hash-style=sysv ../..../intel64/runtime/pincrt/crtbeginS.o -Wl
,-Bsymbolic -Wl,--version-script=../..../source/include/pin/pintool.ver -fabi-ve
rsion=2 -o obj-intel64/inscount0.so obj-intel64/inscount0.o -L../..../intel64/
runtime/pincrt -L../..../intel64/lib -L../..../intel64/lib-ext -L../..../extra
s/xed-intel64/lib -lpin -lxed ../..../intel64/runtime/pincrt/crtendS.o -lpindwar
f -ldwarf -ldl-dynamic -nostdlib -lc++ -lc++abi -lm-dynamic -lc-dynamic -lunwind-
dynamic
make -C ../..../source/tools/Utils dir obj-intel64/cp-pin.exe
make[1]: Entering directory '/home/kali/Downloads/pin/source/tools/Utils'
mkdir -p obj-intel64/
g++ -DTARGET_IA32E -DHOST_IA32E -DFUND_TC_TARGETCPU=FUND_CPU_INTEL64 -DFUND_TC_HO
STCPU=FUND_CPU_INTEL64 -DTARGET_LINUX -DFUND_TC_TARGETOS=FUND_OS_LINUX -DFUND_TC_
HOSTOS=FUND_OS_LINUX -I../..../source/tools/Utils -O3 -std=c++11 -o obj-intel64
/cp-pin.exe cp-pin.cpp -no-pie
make[1]: Leaving directory '/home/kali/Downloads/pin/source/tools/Utils'
../..../pin -t obj-intel64/inscount0.so -- ../..../source/tools/Utils/obj-int
el64/cp-pin.exe makefile obj-intel64/inscount0.makefile.copy \
> obj-intel64/inscount0.out 2>&1
cmp makefile obj-intel64/inscount0.makefile.copy
rm obj-intel64/inscount0.makefile.copy
rm obj-intel64/inscount0.out

```

- 编写并编译测试程序:

再次编写 hello world! 的 C 语言文件并命名为 FirstCpp.c (可复用之前的文件), 使用 gcc -o FirstCpp FirstCpp.c 指令编译成可执行文件 FirstCpp。

```

#include <stdio.h>

void main()
{
    printf("hello world!");
}

```

First FirstCpp FirstCpp.c

```

(kali@kali)-[~/Downloads/testCPP]
$ gcc -o First FirstCpp.c

```

- 执行并获取计数结果:

使用生成的 inscount0.so 执行可执行文件 FirstCpp, 命令为 ./pin /source/tools/ManualExamples/obj-intel64/inscount0.so /testCPP/FirstCpp, 得到插桩后的计数结果 count 192211

```

(kali@kali)-[~/Downloads/pin]
$ ./pin -t ../source/tools/ManualExamples/obj-intel64/inscount0.so -- ../testCP
P/FirstCpp
hello world!

```

```

File Edit Search View Document Help
Count 192211

```


- 代码修改与重新编译:

修改 inscount0.cpp 代码得到新的 tool inscount00.cpp, 新代码中在 VOID Instruction(INS ins, VOID *v) 函数里添加条件判断, 当指令满足一系列编译参数重新编译生成动态链接库 inscount00.so。

```
VOID Instruction(INS ins, VOID *v)
{
    if (INS_Opcode(ins) == XED_ICLASS_MOV &&
        INS_IsMemoryRead(ins) &&
        INS_OperandIsReg(ins, 0) &&
        INS_OperandIsMemory(ins, 1))
    {
        icount++;
    }
}
```

```
(kali@kali) [~/pin/source/tools/manualExamples]
$ make inscount00.test TARGET=intel64
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exceptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -fPIC -DTARGET_LINUX -fabi-version=2 -faligned-new -I../..../source/include/pin -I../..../source/include/pin/gen -isystem /home/kali/Downloads/pin/extras/cxx/include -isystem /home/kali/Downloads/pin/extras/crt/include -isystem /home/kali/Downloads/pin/extras/crt/include/arch-x86_64 -isystem /home/kali/Downloads/pin/extras/crt/include/kernel/uapi -isystem /home/kali/Downloads/pin/extras/crt/include/kernel/uapi/asm-x86 -I../..../extras/components/include -I../..../extras/xed-intel64/include/xed -I../..../source/tools/Utils -I../..../source/tools/InstLib -O3 -fomit-frame-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o obj-intel64/inscount00.o inscount00.cpp
g++ -shared -Wl,-hash-style=sysv ../..../intel64/runtime/pincrt/crtbeginS.o -Bsymbolic -Wl,-version-script=../..../source/include/pin/pintool.ver -fabi-version=2 -o obj-intel64/inscount00.so obj-intel64/inscount00.o -L../..../intel64/runtime/pincrt -L../..../intel64/lib -L../..../intel64/lib-ext -L../..../extras/xed-intel64/lib -lpin -lxd ../..../intel64/runtime/pincrt/crtendS.o -Wl,-pindwarf -ldwarf -ldl -dynamic -nostdlib -lc++ -lc++abi -lm -dynamic -lc -dynamic -lunwind -dynamic
../..../pin -t obj-intel64/inscount00.so -- ../..../source/tools/Utils/cp-intel64/cp-pin.exe makefile obj-intel64/inscount00.makefile.copy \
> obj-intel64/inscount00.out 2>&1
cmp makefile obj-intel64/inscount00.makefile.copy
rm obj-intel64/inscount00.makefile.copy
```

- 执行修改后的程序并获取结果:

执行命令 `./pin -t /source/tools/ManualExamples/obj-intel64/inscount00.so -- ../testCPP/FirstCpp`, 得到修改后的插桩计数 1890。

```
(kali@kali) [~/Downloads/pin]
$ ./pin -t ./source/tools/ManualExamples/obj-intel64/inscount00.so -- ../testCPP/FirstCpp
hello world!

File Edit Search View Document Help
Count 1890
```

心得体会:

本次实验通过复现 malloctrace 和 inscount0 两个 Pin 插桩工具, 让我深入理解了动态二进制插桩技术的核心原理和操作流程。

在 malloctrace 实验中, 我成功捕获了程序运行时 malloc 和 free 函数的调用信息, 包括内存分配大小和返回地址, 这让我对程序的内存管理机制有了更直观的认识。通过分析这些数据, 我学会了如何通过插桩技术监控程序行为, 为后续的软件调试和性能优化打下了基础。

在 inscount0 实验部分, 我不仅复现了基础指令计数功能, 还通过修改代码实现

了对特定指令的筛选统计。这一过程让我掌握了 Pin 工具的灵活性和可扩展性，同时也加深了对指令级插桩的理解。实验中的代码修改和调试环节锻炼了我的问题解决能力，尤其是在处理编译错误和逻辑条件时，学会了如何查阅文档和调整参数。

总的来说，本次实验让我认识到插桩技术在软件安全分析、性能优化和漏洞检测中的重要作用。通过实践，我熟悉了 Linux 环境下的开发流程，并掌握了 Pin 工具的基本使用方法。